MANUAL TÉCNICO

Sistema web SPARROW para la administración de toallas en microfibra

Arquitectura, instalación, base de datos, seguridad, pruebas y mantenimiento

Campo	Información
Programa	Técnico en Programación de Software
Proyecto	Desarrollo de una aplicación web para la administración de inventario, pedidos y proveedores de toallas en microfibra (SPARROW)
Integrantes	Danna Carrillo - Angel gutierrez
Institución	Colegio Técnico Palermo I.E.D. – SENA
Versión del manual	1.0
Fecha	2026

CONTENIDO

1. Propósito del manual técnico
2. Identificación técnica del producto
3. Arquitectura del sistema
4. Módulos funcionales implementados
5. Requisitos técnicos del entorno
6. Instalación y configuración del ambiente
7. Estructura del código fuente
8. Base de datos
9. Requerimientos no funcionales y seguridad
10. Pruebas técnicas
11. Despliegue y puesta en funcionamiento
12. Respaldo y recuperación
13. Mantenimiento técnico
14. Solución de problemas frecuentes
15. Documentación del código
16. Control de versiones y entrega técnica
17. Lista de verificación antes de entregar
18. Anexos técnicos

> **Base de elaboración:** Este manual se elaboró a partir de la lectura completa del código fuente entregado (carpeta `sparrow`: 55 archivos de código: 52 PHP, 1 JavaScript, 1 CSS y 1 SQL). Todo lo descrito corresponde a lo implementado; los puntos que dependen del equipo donde se instale o ejecute el sistema (versiones exactas, resultados de pruebas) se dejan marcados como **[Completar]** o **Pendiente**.

1. Propósito del manual técnico

Este manual documenta cómo está construido, configurado, instalado, probado, desplegado y mantenido el sistema SPARROW, de modo que otro desarrollador o técnico pueda comprenderlo y ponerlo en funcionamiento sin depender de explicaciones verbales del equipo.

SPARROW es una aplicación web que permite a cada usuario registrado administrar su **inventario** de toallas en microfibra, sus **pedidos** y sus **proveedores**, además de ofrecer páginas públicas de información y un formulario de contacto. Está desarrollada con **PHP estructurado (MySQLi)** y **MySQL**, organizada por capas, sin frameworks, sin programación orientada a objetos y sin PDO.

> **Importante:** Se documentan únicamente las tecnologías realmente utilizadas en el código: HTML5, CSS3, JavaScript, PHP y MySQL (servidor Apache de XAMPP). No se usan Python, Flutter, frameworks ni librerías externas.

1.1 Alcance técnico documentado

- Arquitectura general y relación entre interfaz, lógica, acceso a datos, base de datos y servidor.
- Requisitos de software y hardware, e instalación del ambiente desde cero.
- Estructura del proyecto: carpetas, archivos principales y responsabilidades.
- Modelo de datos, tablas, claves, relaciones y script de creación.
- Módulos funcionales y flujo técnico de las operaciones críticas.
- Seguridad, autenticación, control de acceso, respaldo y recuperación.
- Plan de pruebas técnicas, despliegue, mantenimiento y solución de fallas.
- Control de versiones, entregables y anexos.

1.2 Convenciones del documento

Convención	Significado
`Texto monoespaciado`	Nombre de archivo, carpeta, función, constante, tabla o comando.
[Completar]	Dato que depende del equipo o del entorno real y debe registrarse al instalar o desplegar (por ejemplo, la versión exacta de PHP).
Pendiente	Prueba o verificación que debe ejecutarse y registrarse; el resultado esperado se deduce del código.
Rutas	Se expresan relativas a la raíz del proyecto (`sparrow/`).

2. Identificación técnica del producto

Elemento	Información
Nombre del sistema	SPARROW (título de página: «Mi Proyecto de Grado - SPARROW»).
Versión	1.0.0

Elemento	Información
Tipo de aplicación	Aplicación web responsiva (interfaz adaptable con regla `@media (max-width: 868px)`), renderizada en el servidor.
Usuarios	Visitante (sin sesión) y usuario registrado. No hay roles diferenciados; cada usuario solo accede a sus propios registros.
URL o ambiente	Local: `http://localhost/sparrow/` (XAMPP). [Completar con la URL del servidor de pruebas o publicada, si existe].
Repositorio	[Completar con el enlace al repositorio, si existe]. Entrega actual: archivo `sparrow.zip`.
Base de datos	MySQL/MariaDB, base `sparrow_db`, codificación `utf8mb4`, motor InnoDB, 5 tablas.
Lenguajes	PHP (servidor, MySQLi procedural/preparado), JavaScript (comodidades de interfaz), HTML5, CSS3, SQL.
Responsables	[Completar integrantes y responsabilidad técnica de cada uno].

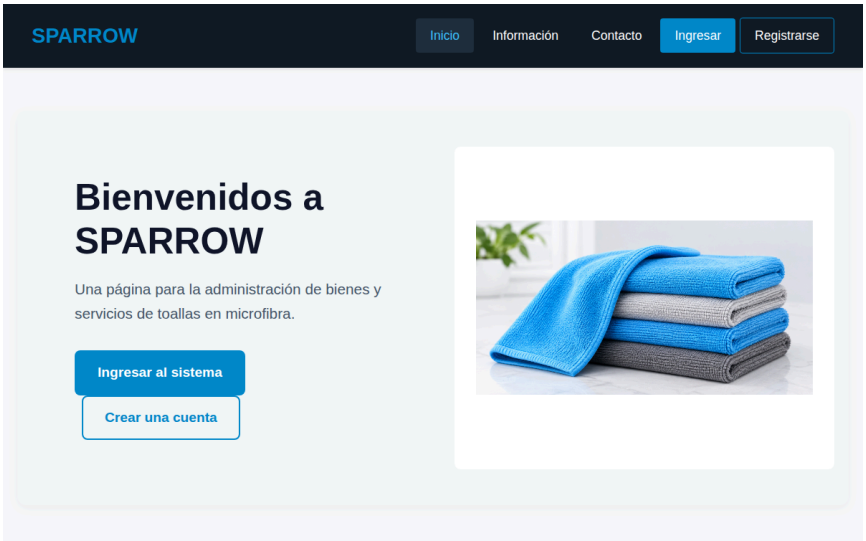


Figura 1. Pantalla principal del sistema (versión 1.0.0).

3. Arquitectura del sistema

SPARROW sigue una **arquitectura por capas** con separación de responsabilidades (config · datos · lógica · procesos · vistas), sin patrón MVC formal. El navegador solicita páginas a Apache/PHP; `index.php` actúa como punto de entrada único para las pantallas, y los formularios envían sus datos (POST) directamente a los archivos de `procesos/`, que validan, consultan la base de datos mediante los DAO y redirigen de vuelta a `index.php`.

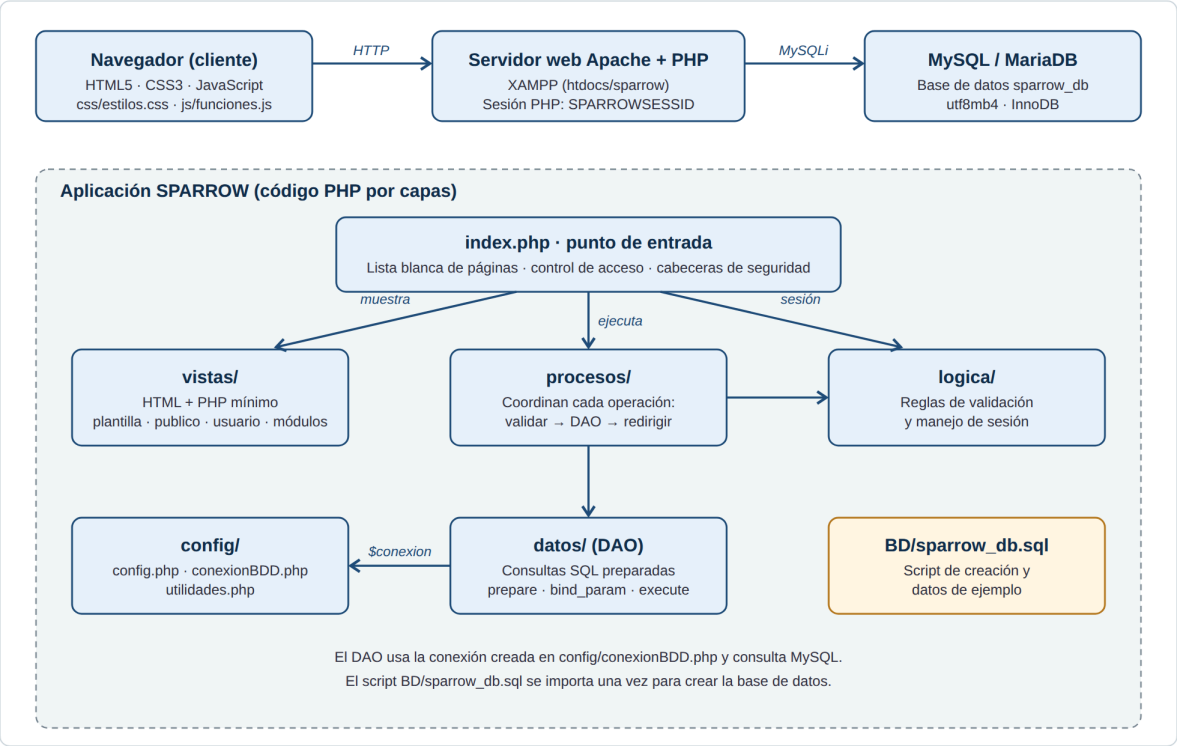


Figura 2. Diagrama de componentes y capas de SPARROW.

Componente	Descripción técnica
Cliente / Front-end	HTML5 generado por PHP; estilos en `css/estilos.css` (116 líneas); `js/funciones.js` (43 líneas) agrega comodidades: bloqueo de números en campos `data-sin-numeros`, confirmación de eliminación con `data-confirmar`, comparación de contraseñas y cierre de mensajes con clic. **La validación real siempre se repite en PHP.**
Back-end / lógica	PHP estructurado. `logica/` contiene las reglas de validación (`validarUsuario`, `validarInventario`, `validarPedido`, `validarProveedor`, `validarContacto`) y el manejo de sesión (`sesion.php`). `procesos/` coordina cada operación.
Acceso a datos	Capa DAO en `datos/` con funciones que ejecutan SQL **preparado** (`prepare` + `bind_param` + `execute`). No contiene HTML ni validaciones.
Base de datos	MySQL, base `sparrow_db`; conexión creada en `config/conexionBDD.php` con MySQLi y conjunto de caracteres `utf8mb4`.
Servidor / hosting	Apache (XAMPP) con PHP, carpeta `htdocs/sparrow`. Los directorios internos se protegen con `.htaccess` (`Require all denied`).
Servicios externos	Ninguno. No hay APIs, correo ni almacenamiento externo.

3.1 Recorrido de una solicitud

Lectura (ejemplo: `index.php?pagina=inventario`): `index.php` consulta la lista blanca de páginas, comprueba la sesión, ejecuta el proceso `listarInventario.php` (que invoca al DAO y deja las

variables `$resultado` y `$total`) y finalmente carga las vistas: encabezado, mensaje, banner de usuario, formulario, lista y pie.

Escritura (ejemplo: guardar inventario): el formulario envía un POST a `procesos/inventario/guardarInventario.php`, que exige sesión, valida los datos, solicita al DAO el `INSERT` preparado y redirige a `index.php?pagina=inventario&mensaje=...&tipo=...` (patrón Post/Redirect/Get, que evita reenvíos al recargar).

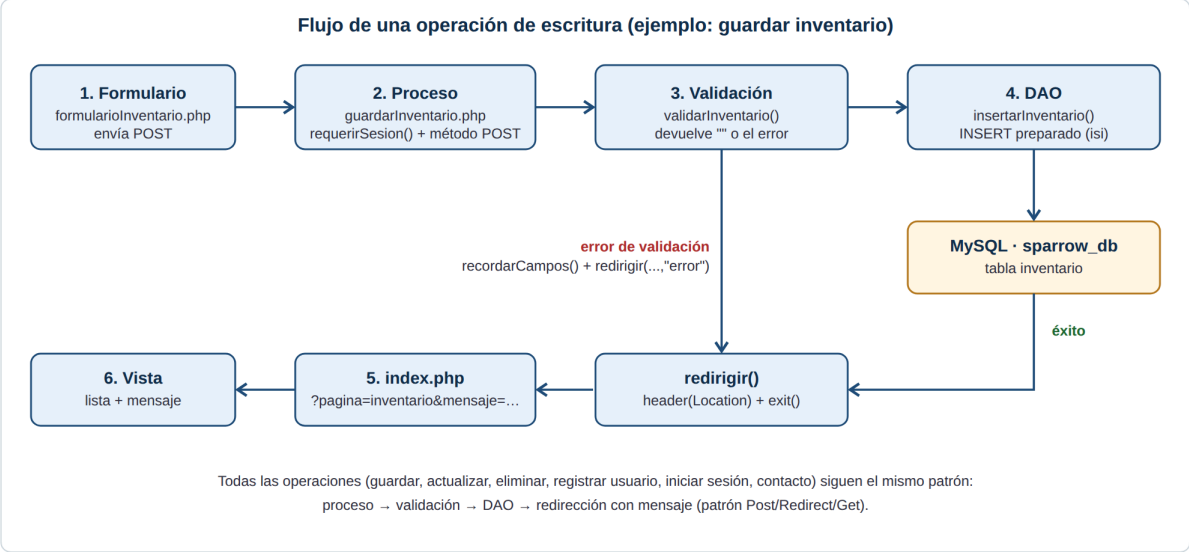


Figura 3. Flujo de una operación de escritura.

3.2 Enrutamiento y control de acceso en `index.php`

`index.php` define un arreglo `$paginas` (lista blanca) donde cada página declara si es `privada` (requiere sesión), `invitado` (solo sin sesión), su `proceso` opcional y las `vistas` que se muestran. Cualquier valor de `?pagina=` que no esté en la lista se reemplaza por `inicio`.

Página (?pagina=)	Acceso	Proceso previo	Vistas
`inicio`, `informacion`, `contacto`	Pública	—	`publico/inicio.php`, `publico/informacion.php`, `publico/contacto.php`
`login`, `registro`	Invitado	—	`usuario/formularioLogin.php`, `usuario/formularioRegistro.php`
`inventario`	Privada	`listarInventario.php`	`formularioInventario.php`, `listaInventario.php`
`inventario-editar`	Privada	`editarInventario.php`	`formularioEditar.php`
`pedidos` / `pedidos-editar`	Privada	`listarPedidos.php` / `editarPedido.php`	Equivalentes en `vistas/pedido/`

Página (?pagina=)	Acceso	Proceso previo	Vistas
`proveedores` / `proveedores-editar`	Privada	`listarProveedores.php` / `editarProveedor.php`	Equivalentes en `vistas/proveedor/`

4. Módulos funcionales implementados

La tabla resume los módulos presentes en el código. El estado se basa en la existencia del flujo completo (vista, proceso, validación y DAO); su verificación en ejecución se registra en la sección 10.

Código	Módulo / función	Rol	Estado	Evidencia técnica
MOD-01	Registro de usuarios	Visitante	Implementado	`formularioRegistro.php` · `validarRegistro()` · `registrarUsuario.php` · tabla `usuarios`
MOD-02	Inicio de sesión	Visitante	Implementado	`formularioLogin.php` · `iniciarSesion.php` · `password_verify` · sesión `SPARROWSESSID`
MOD-03	Cierre de sesión	Usuario	Implementado	`cerrarSesion.php` (solo POST) · `session_destroy()`
MOD-04	Gestión de inventario (CRUD)	Usuario	Implementado	`procesos/inventario/*` · `inventarioDAO.php` · tabla `inventario`
MOD-05	Gestión de pedidos (CRUD)	Usuario	Implementado	`procesos/pedido/*` · `pedidoDAO.php` · tabla `pedidos`
MOD-06	Gestión de proveedores (CRUD)	Usuario	Implementado	`procesos/proveedor/*` · `proveedorDAO.php` · tabla `proveedores`
MOD-07	Formulario de contacto	Visitante / Usuario	Implementado (solo escritura)	`guardarContacto.php` · `contactoDAO.php` · tabla `mensajes_contacto`
MOD-08	Páginas informativas	Público	Implementado	`publico/inicio.php`, `publico/informacion.php`
—	Recuperación de contraseña, perfil, roles, reportes, consulta de mensajes de contacto	—	No implementado	No existe código asociado en la versión 1.0.

4.1 Descripción técnica por módulo

MOD-01 · Registro de usuarios

Campo	Contenido
Objetivo técnico	Crear cuentas nuevas garantizando datos válidos, usuario y correo únicos y contraseña cifrada.

Campo	Contenido
Entradas	`txtNombre`, `txtUsuario`, `txtCorreo`, `txtPassword`, `txtPassword2` (POST).
Proceso	1) `validarRegistro()`; 2) `existeUsuario()` y `existeCorreo()`; 3) `password_hash($password, PASSWORD_DEFAULT)`; 4) `insertarUsuario()`.
Salidas	Redirección a `login` con «Cuenta creada correctamente. Ahora puedes iniciar sesión.» o a `registro` con el error.
Archivos implicados	`vistas/usuario/formularioRegistro.php`, `procesos/usuario/registrarUsuario.php`, `logica/validarUsuario.php`, `datos/usuario/usuarioDAO.php`.
Tablas implicadas	`usuarios` (SELECT por `usuario` y `correo`; INSERT).
Reglas de validación	Nombre 3–80 (letras, espacios, `.`, `'`, `-`); usuario `^[A-Za-z0-9_]{4,30}$`; correo válido ≤ 100; contraseña 6–72 con al menos una letra y un número; confirmación idéntica.
Errores controlados	Campos vacíos, formato inválido, usuario/correo duplicado, fallo de inserción («No fue posible crear la cuenta. Intenta de nuevo.»). Se conservan los campos escritos, **nunca** las contraseñas.

MOD-02 y MOD-03 · Inicio y cierre de sesión

Campo	Contenido
Objetivo técnico	Autenticar al usuario por nombre de usuario o correo y mantener su sesión; cerrarla de forma segura.
Entradas	`txtLogin`, `txtPassword` (POST). El cierre se solicita con un formulario POST (botón «Salir»).
Proceso	`validarLogin()` → `buscarUsuarioPorLogin()` (`WHERE usuario = ? OR correo = ?`) → `password_verify()` → `session_regenerate_id(true)` → se guardan `usuario_id`, `usuario` y `nombre` en `$_SESSION`. El cierre vacía `$_SESSION`, expira la cookie y llama a `session_destroy()`.
Salidas	Redirección a `inventario` (éxito) o a `login` con mensaje único «Este usuario no está registrado o la contraseña es incorrecta.».
Archivos implicados	`procesos/usuario/iniciarSesion.php`, `cerrarSesion.php`, `logica/sesion.php`, `logica/validarUsuario.php`, `datos/usuario/usuarioDAO.php`.
Tablas implicadas	`usuarios` (SELECT).
Errores controlados	Campos vacíos; credenciales inválidas (mismo mensaje en ambos casos); acceso por método distinto de POST (redirige).

MOD-04, MOD-05 y MOD-06 · CRUD de inventario, pedidos y proveedores

Los tres módulos comparten exactamente el mismo patrón (listar · guardar · editar · actualizar · eliminar). Solo cambian los campos:

Módulo	Tabla / DAO	Campos y validación	Archivos de proceso
Inventario	`inventario` / `inventarioDAO.php`	`detalle` 3–150; `cantidad` entero 0–1.000.000	`listarInventario`, `guardarInventario`, `editarInventario`, `actualizarInventario`, `eliminarInventario`
Pedidos	`pedidos` / `pedidoDAO.php`	`cliente` 3–150; `descripcion` 3–255	`listarPedidos`, `guardarPedido`, `editarPedido`, `actualizarPedido`, `eliminarPedido`
Proveedores	`proveedores` / `proveedorDAO.php`	`nombre` 3–150; `contacto` 5–100	`listarProveedores`, `guardarProveedor`, `editarProveedor`, `actualizarProveedor`, `eliminarProveedor`

Campo	Contenido
Objetivo técnico	Permitir a cada usuario crear, consultar, modificar y eliminar sus propios registros.
Entradas	Campos del formulario (POST) y `id` del registro (oculto en POST; en `GET` para abrir la edición).
Proceso	**Guardar:** `requerirSesion()` → método POST → validación → `insertar...()`. **Editar:** `filter_var($_GET["id"], FILTER_VALIDATE_INT)` → `buscar...($id, $usuarioId)`. **Actualizar:** valida `id`, comprueba pertenencia, **revalida** y ejecuta `actualizar...()`. **Eliminar:** solo POST, comprueba pertenencia y ejecuta `eliminar...()`.
Salidas	Redirección a la lista con mensaje de éxito o error (por ejemplo, «Registro guardado en el inventario.»).
Control de propiedad	Todas las consultas incluyen `usuario_id = ?`; un `id` ajeno produce «El registro no existe.».
Errores controlados	ID no válido; registro inexistente o ajeno; validación fallida (se vuelve al formulario con los valores escritos); fallo del DAO («No fue posible guardar/actualizar/eliminar el registro.»).

MOD-07 · Formulario de contacto

Campo	Contenido
Objetivo técnico	Recibir mensajes de visitantes y almacenarlos.
Entradas	`txtNombre` (3–100, sin números), `txtCorreo` (válido ≤ 100), `txtMensaje` (3–1000).
Proceso	`validarContacto()` → `insertarMensaje()` (INSERT preparado).
Salidas	«Mensaje enviado exitosamente.» o mensaje de error. No se envía correo.
Archivos implicados	`vistas/publico/contacto.php`, `procesos/contacto/guardarContacto.php`, `logica/validarContacto.php`, `datos/contacto/contactoDAO.php`.

Campo	Contenido
Tablas implicadas	`mensajes_contacto` (INSERT). No existe pantalla de consulta; se lee desde phpMyAdmin o SQL.

5. Requisitos técnicos del entorno

5.1 Requisitos de software

Las versiones mínimas se **deducen del código** (por ejemplo, el arreglo de opciones en `session_set_cookie_params` exige PHP 7.3 o superior y `get_result()` requiere el driver `mysqlnd`). La versión exacta instalada y probada debe registrarse en la columna correspondiente.

Software	Versión mínima / exacta	Uso dentro del proyecto	Obligatorio
Sistema operativo	Windows 10/11 (documentado en README) · [Completar versión exacta]	Desarrollo y servidor local	Obligatorio (también sirve Linux/macOS con LAMP/MAMP)
XAMPP (Apache + MySQL/MariaDB + PHP + phpMyAdmin)	Recomendado con PHP 8.x · [Completar versión exacta]	Servidor web, base de datos y gestor de BD	Obligatorio (o stack equivalente)
PHP	≥ 7.3 (recomendado 8.x) · [Completar]	Lógica del servidor	Obligatorio
Extensiones PHP	`mysqli` (con `mysqlnd`), `mbstring`, `filter`, `session`	Conexión, longitud UTF-8, validación, sesiones	Obligatorias (activas por defecto en XAMPP)
Apache	2.4 (directiva `Require all denied`) · `AllowOverride` habilitado	Servidor web y protección con `.htaccess`	Obligatorio
MySQL / MariaDB	MySQL ≥ 5.7 o MariaDB ≥ 10.x con `utf8mb4` e InnoDB · [Completar]	Persistencia de datos	Obligatorio
HTML5 / CSS3 / JavaScript	Sin dependencias externas	Interfaz web	Incluido en el proyecto
Editor o IDE	[Completar: VS Code, PhpStorm, etc.]	Edición y depuración	Opcional
Navegadores probados	[Completar: Chrome, Firefox, Edge…]	Verificación de compatibilidad	Obligatorio documentarlo

Dependencias de terceros: ninguna. El proyecto no usa Composer, npm, frameworks CSS/JS ni CDN; todo el CSS y JavaScript es propio.

5.2 Requisitos de hardware

Recurso	Mínimo (referencia)	Recomendado	Observaciones
Procesador	Doble núcleo 1,6 GHz	Cuatro núcleos 2,0 GHz o más	Aplicación liviana; valores de referencia para XAMPP. [Validar con el equipo real]
Memoria RAM	2 GB	4 GB o más	Apache + MySQL + navegador en el mismo equipo.
Almacenamiento	1 GB libre (XAMPP + proyecto)	5 GB (con respaldos)	El proyecto pesa ~2,4 MB (la imagen de portada `img/imagen.jpg` ocupa ~2,3 MB).
Conectividad	Red local	Red local o Internet	Necesaria para que otros dispositivos accedan.
Dispositivo cliente	PC, tablet o celular con navegador moderno	Ídem	Interfaz adaptable a pantallas pequeñas.

> **Nota:** Estos valores son referenciales para un entorno escolar o de hosting básico; no corresponden a mediciones realizadas. Se recomienda registrar los datos reales del equipo en que se pruebe el sistema.

6. Instalación y configuración del ambiente

Procedimiento para instalar SPARROW desde cero en Windows con XAMPP (el mismo descrito en el `README.md` del proyecto).

> **Evidencias:** El equipo debe incorporar capturas numeradas de los pasos 6.2, 6.6 y 6.8 (panel de XAMPP, importación en phpMyAdmin y pantalla de inicio) al ejecutar el procedimiento, con la versión y fecha de la evidencia.

6.1 Preparación del equipo

Equipo con Windows 10/11, permisos de administrador para instalar software y sin otro servicio usando los puertos 80 (Apache) y 3306 (MySQL), o bien se deberán cambiar los puertos.

6.2 Instalación del entorno de ejecución

1. Descargar XAMPP desde el sitio oficial (apachefriends.org) con PHP 8.x.
2. Ejecutar el instalador y seleccionar como mínimo **Apache**, **MySQL**, **PHP** y **phpMyAdmin**. Ruta por defecto: `C:\xampp`.
3. Abrir el **Panel de control de XAMPP** e iniciar los módulos **Apache** y **MySQL** (ambos deben mostrarse en verde).

6.3 Obtención del código fuente

Descomprimir `sparrow.zip` y copiar la carpeta del proyecto dentro de `C:\xampp\htdocs\` de modo que el archivo de entrada quede en:

```
C:\xampp\htdocs\sparrow\index.php
```

> **Cuidado:** El .zip entregado contiene una carpeta `sparrow` dentro de otra `sparrow`. Debe copiarse la carpeta interna (la que contiene `index.php`); de lo contrario la URL será `http://localhost/sparrow/sparrow/`.

6.4 Instalación de dependencias

No hay dependencias que instalar (sin `composer.json`, `package.json` ni `requirements.txt`). Solo se debe comprobar que las extensiones `mysqli`, `mbstring` y `filter` estén activas; en XAMPP lo están por defecto (se puede verificar en `http://localhost/dashboard/phpinfo.php`).

6.5 Configuración de variables

La configuración está en `config/config.php` mediante constantes. No se usa archivo `.env`.

Constante	Valor por defecto	Descripción
`SERVIDOR`	`localhost`	Servidor donde corre MySQL.
`USUARIO`	`root`	Usuario de MySQL (en XAMPP local es `root`).
`PASSWORD`	`""` (vacía)	Contraseña de MySQL. En producción debe definirse una contraseña robusta y un usuario con privilegios limitados.
`BASE_DATOS`	`sparrow_db`	Nombre de la base de datos creada por el script.
`URL_BASE`	Calculada	Ruta web del proyecto, derivada de la ubicación real de `config.php` y de `DOCUMENT_ROOT`. Permite renombrar la carpeta sin tocar el código.

Ejemplo de configuración para un servidor con contraseña (valores ficticios):

```
define("SERVIDOR",   "localhost");
define("USUARIO",    "sparrow_user");
define("PASSWORD",   "CambiarEstaClave#2026");
define("BASE_DATOS", "sparrow_db");
```

6.6 Configuración de la base de datos

1. Abrir `http://localhost/phpmyadmin`.
2. Ir a la pestaña **Importar**, elegir el archivo `BD/sparrow_db.sql` y pulsar **Importar**.
3. El script crea la base `sparrow_db`, sus 5 tablas, el usuario de demostración y datos de ejemplo.

Alternativa por línea de comandos:

```
C:\xampp\mysql\bin\mysql.exe -u root -p < BD\sparrow_db.sql
```

> **Advertencia:** El script ejecuta `DROP TABLE IF EXISTS` sobre todas las tablas. **Reimportarlo borra todos los datos existentes.** Haga un respaldo antes (sección 12).

6.7 Inicio del sistema

Con Apache y MySQL en ejecución, abrir en el navegador:

```
http://localhost/sparrow/

# Si Apache usa otro puerto (por ejemplo 8080):
http://localhost:8080/sparrow/
```

6.8 Verificación inicial

- Se muestra la pantalla de inicio «Bienvenidos a SPARROW» con su imagen.
- No aparece el mensaje «No fue posible conectar con la base de datos…».
- Iniciar sesión con la cuenta de demostración (usuario `admin`, contraseña indicada en el `README.md`) abre la pantalla de Inventario con 3 registros de ejemplo.
- Pedidos muestra 1 pedido y Proveedores muestra 1 proveedor de ejemplo.
- Crear un registro de prueba y recargar la página confirma la escritura en la base de datos.

7. Estructura del código fuente

```
sparrow/
├── index.php             punto de entrada: lista blanca, acceso, vistas
├── config/
│   ├── config.php        constantes: SERVIDOR, USUARIO, PASSWORD, BASE_DATOS
│   ├── conexionBDD.php   crea la conexión $conexion (MySQLi, utf8mb4)
│   └── utilidades.php    redirigir(), recordarCampos(), recuperarFormulario()
├── datos/                DAO: solo SQL preparado
│   ├── usuario/usuarioDAO.php
│   ├── inventario/inventarioDAO.php
│   ├── pedido/pedidoDAO.php
│   ├── proveedor/proveedorDAO.php
│   └── contacto/contactoDAO.php
├── logica/               validaciones y sesión
│   ├── sesion.php
│   └── validarUsuario / validarInventario / validarPedido /
│       validarProveedor / validarContacto .php
├── procesos/             validar → DAO → redirigir
│   ├── usuario/     registrarUsuario, iniciarSesion, cerrarSesion
│   ├── inventario/  listar, guardar, editar, actualizar, eliminar
│   ├── pedido/      listar, guardar, editar, actualizar, eliminar
│   ├── proveedor/   listar, guardar, editar, actualizar, eliminar
│   └── contacto/    guardarContacto
├── vistas/               HTML + PHP mínimo
│   ├── plantilla/   encabezado, mensaje, bannerUsuario, pie
```

```
│   ├── publico/     inicio, informacion, contacto
│   ├── usuario/     formularioLogin, formularioRegistro
│   └── inventario/, pedido/, proveedor/
│                    formulario, formularioEditar, lista
├── css/estilos.css
├── js/funciones.js
├── img/imagen.jpg
└── BD/sparrow_db.sql
```

Figura 4. Árbol de carpetas del proyecto.

Carpeta / archivo	Responsabilidad	Archivos	Líneas aprox.
`/` (raíz)	Punto de entrada y documentación	`index.php`, `README.md`	132 (index)
`config/`	Parámetros, conexión y utilidades	3 PHP + `.htaccess`	104
`datos/`	Acceso a datos (DAO)	5 PHP + `.htaccess`	394
`logica/`	Reglas de validación y sesión	6 PHP + `.htaccess`	253
`procesos/`	Coordinación de operaciones	19 PHP	582
`vistas/`	Interfaz HTML	18 PHP + `.htaccess`	461
`css/`, `js/`, `img/`	Estilos, comportamiento del cliente e imagen	3 archivos	116 · 43 · —
`BD/`	Script de creación y carga	`sparrow_db.sql` + `.htaccess`	88

Los directorios `config`, `datos`, `logica`, `vistas` y `BD` incluyen un `.htaccess` con `Require all denied`, que impide abrir sus archivos directamente desde el navegador. `procesos/` no lo incluye porque los formularios envían sus datos directamente a esos archivos.

8. Base de datos

La base `sparrow_db` (`utf8mb4`, `utf8mb4_general_ci`, motor InnoDB) contiene cinco tablas. Las tres tablas de negocio dependen de `usuarios` mediante la clave foránea `usuario_id`, lo que permite aislar la información de cada usuario.

8.1 Modelo entidad-relación final

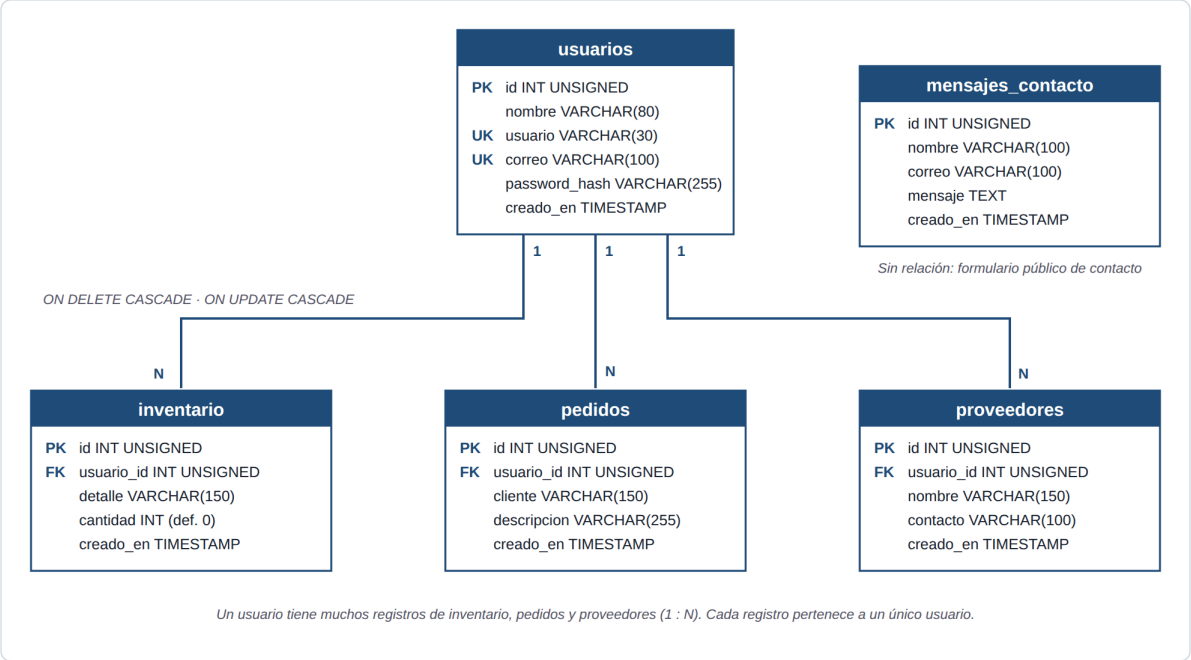


Figura 5. Modelo entidad-relación de sparrow_db.

Relación	Cardinalidad	Regla de integridad
`usuarios` → `inventario`	1 : N	FK `fk_inventario_usuario`; `ON DELETE CASCADE ON UPDATE CASCADE`.
`usuarios` → `pedidos`	1 : N	FK `fk_pedidos_usuario`; `ON DELETE CASCADE ON UPDATE CASCADE`.
`usuarios` → `proveedores`	1 : N	FK `fk_proveedores_usuario`; `ON DELETE CASCADE ON UPDATE CASCADE`.
`mensajes_contacto`	Sin relación	Tabla independiente: los mensajes los envía cualquier visitante.

Al eliminar un usuario se eliminan en cascada todos sus registros de inventario, pedidos y proveedores. Los índices `idx_*_usuario` aceleran las consultas filtradas por usuario.

8.2 Diccionario de datos técnico

Tabla	Campo	Tipo	Long.	PK	FK	Nulo	Descripción / regla
usuarios	id	INT UNSIGNED	—	Sí	No	No	Identificador único autoincremental.
usuarios	nombre	VARCHAR	80	No	No	No	Nombre completo.

Tabla	Campo	Tipo	Long.	PK	FK	Nulo	Descripción / regla
usuarios	usuario	VARCHAR	30	No	No	No	Nombre de usuario. Único (`uq_usuario`).
usuarios	correo	VARCHAR	100	No	No	No	Correo. Único (`uq_correo`).
usuarios	password_hash	VARCHAR	255	No	No	No	Hash `password_hash()` (bcrypt).
usuarios	creado_en	TIMESTAMP	—	No	No	No	Por defecto `CURRENT_TIMESTAMP`.
inventario	id	INT UNSIGNED	—	Sí	No	No	Identificador único autoincremental.
inventario	usuario_id	INT UNSIGNED	—	No	Sí	No	Propietario → `usuarios.id`.
inventario	detalle	VARCHAR	150	No	No	No	Tipo/descripción de la toalla.
inventario	cantidad	INT	—	No	No	No	Unidades. Por defecto 0.
inventario	creado_en	TIMESTAMP	—	No	No	No	Fecha de creación.
pedidos	id	INT UNSIGNED	—	Sí	No	No	Identificador único autoincremental.
pedidos	usuario_id	INT UNSIGNED	—	No	Sí	No	Propietario → `usuarios.id`.
pedidos	cliente	VARCHAR	150	No	No	No	Cliente o destino.
pedidos	descripcion	VARCHAR	255	No	No	No	Detalle del pedido.
pedidos	creado_en	TIMESTAMP	—	No	No	No	Fecha de creación.
proveedores	id	INT UNSIGNED	—	Sí	No	No	Identificador único autoincremental.
proveedores	usuario_id	INT UNSIGNED	—	No	Sí	No	Propietario → `usuarios.id`.
proveedores	nombre	VARCHAR	150	No	No	No	Nombre del proveedor.
proveedores	contacto	VARCHAR	100	No	No	No	Teléfono o dato de contacto.
proveedores	creado_en	TIMESTAMP	—	No	No	No	Fecha de creación.
mensajes_contacto	id	INT UNSIGNED	—	Sí	No	No	Identificador único autoincremental.
mensajes_contacto	nombre	VARCHAR	100	No	No	No	Nombre de quien escribe.
mensajes_contacto	correo	VARCHAR	100	No	No	No	Correo de quien escribe.
mensajes_contacto	mensaje	TEXT	—	No	No	No	Contenido (hasta 1000 caracteres validados).
mensajes_contacto	creado_en	TIMESTAMP	—	No	No	No	Fecha de creación.

8.3 Script de creación y carga

El archivo `BD/sparrow_db.sql` (88 líneas) se ejecuta en este orden:

1. `CREATE DATABASE IF NOT EXISTS sparrow_db` con `utf8mb4` / `utf8mb4_general_ci` y `USE sparrow_db`.
2. `DROP TABLE IF EXISTS` en orden inverso a las dependencias: `mensajes_contacto`, `proveedores`, `pedidos`, `inventario`, `usuarios`.
3. `CREATE TABLE` de `usuarios`, `inventario`, `pedidos`, `proveedores` y `mensajes_contacto`, con claves primarias, únicas, índices y claves foráneas.
4. Datos iniciales: usuario `admin` (correo `admin@sparrow.com`, contraseña almacenada como hash bcrypt), 3 registros de inventario, 1 pedido y 1 proveedor de ejemplo.

8.4 Conexión del sistema a la base de datos

La conexión se crea una sola vez en `config/conexionBDD.php`, que carga las constantes de `config.php`. Los procesos la incluyen con `require_once` y reciben la variable `$conexion`, que pasan a los DAO.

```
<?php

// Carga la configuración con una ruta basada en la ubicación real de este archivo.
require_once __DIR__ . "/config.php";

// Evita que MySQLi muestre errores técnicos al usuario final.
mysqli_report(MYSQLI_REPORT_OFF);

// Crea la conexión con las constantes definidas en config.php.
$conexion = new mysqli(SERVIDOR, USUARIO, PASSWORD, BASE_DATOS);

// Si la conexión falla se detiene el programa con un mensaje claro.
if ($conexion->connect_errno) {
    die("No fue posible conectar con la base de datos. "
      . "Verifica que MySQL esté iniciado en XAMPP y que hayas importado BD/sparrow_db.sql.");
}

// UTF-8 completo para tildes, eñes y caracteres especiales.
$conexion->set_charset("utf8mb4");
```

- `mysqli_report(MYSQLI_REPORT_OFF)` evita que MySQLi muestre errores técnicos al usuario.
- Si la conexión falla se detiene la ejecución con un mensaje genérico, sin revelar credenciales.
- `set_charset("utf8mb4")` garantiza el correcto manejo de tildes, eñes y símbolos.
- Las credenciales están en `config/config.php`; esa carpeta está protegida con `.htaccess`. No deben publicarse contraseñas reales en el repositorio.

9. Requerimientos no funcionales y seguridad

Requisito	Mecanismo implementado	Cómo se verifica	Evidencia
Rendimiento	Consultas simples con índices por `usuario_id`; sin librerías externas; CSS y JS ligeros (116 y 43 líneas).	Tiempo de carga de las listas con datos de prueba.	[Completar]
Escalabilidad	Separación por capas y patrón repetible por módulo; agregar un módulo implica tabla, DAO, proceso, validación y vistas (ver sección 13).	Revisión de diseño; prueba con varios usuarios.	[Completar]
Seguridad	Consultas preparadas, hash de contraseñas, sesión con ID regenerado, escape de salida, filtro por propietario, POST para acciones que modifican datos, `.htaccess`.	Pruebas PT-02, PT-04, PT-05, PT-10, PT-11, PT-18, PT-19.	[Completar]
Disponibilidad / fiabilidad	Mensajes de error controlados; validación en servidor; mensaje claro si falla la conexión a la BD.	Detener MySQL y abrir la aplicación; reiniciar Apache.	[Completar]
Usabilidad	Diseño adaptable (`@media (max-width: 868px)`), mensajes de éxito/error, conservación de datos al fallar la validación, confirmación antes de eliminar.	Prueba en celular y navegador (PT-16).	[Completar]
Respaldo	No automatizado; se realiza con `mysqldump` o phpMyAdmin (sección 12).	Restauración desde copia (PT-17).	[Completar]

9.1 Controles de seguridad implementados

Control	Implementación en el código
Contraseñas	`password_hash($password, PASSWORD_DEFAULT)` al registrar y `password_verify()` al iniciar sesión; nunca se guarda ni recuerda en texto plano. Longitud 6–72 (límite de bcrypt) con letra y número.
Inyección SQL	Todas las consultas con datos del usuario usan `prepare()` + `bind_param()` + `execute()`. No se concatenan variables en el SQL.
XSS	Toda salida de datos se escapa con `htmlspecialchars()` (listas, banner, mensajes y valores de formularios). Los identificadores se convierten con `(int)`.
Autenticación y sesión	Sesión con nombre propio `SPARROWSESSID`, cookie `httponly` y `samesite=Lax`; `session_regenerate_id(true)` al autenticar (evita fijación de sesión); cierre con `session_destroy()` y expiración de cookie.
Control de acceso	Lista blanca de páginas en `index.php`; páginas `privada` exigen sesión y páginas `invitado` redirigen a quien ya inició sesión; los procesos de escritura llaman a `requerirSesion()`.
Aislamiento entre usuarios	Todas las consultas de inventario, pedidos y proveedores incluyen `usuario_id = ?`; se verifica la propiedad antes de editar o eliminar.

Control	Implementación en el código
Acciones que modifican datos	Eliminar y cerrar sesión solo se aceptan por POST (no mediante enlaces).
Validación	En el cliente (`required`, `maxlength`, `pattern`, `min/max`) y **repetida en el servidor** (`logica/validar*.php`).
Cabeceras HTTP	`X-Content-Type-Options: nosniff`, `X-Frame-Options: SAMEORIGIN` y `Cache-Control: no-store` en páginas privadas.
Protección de archivos	`.htaccess` con `Require all denied` en `config`, `datos`, `logica`, `vistas` y `BD`.
Mensajes de error	Mensaje único ante usuario inexistente o clave incorrecta; errores de MySQLi desactivados para el usuario final.

9.2 Observaciones y mejoras recomendadas

Durante la revisión del código se identificaron los siguientes puntos, que no impiden el funcionamiento pero conviene atender antes de un despliegue con datos reales:

Observación	Detalle	Recomendación
Sin tokens CSRF	Los formularios no incluyen token anti-CSRF; la cookie `SameSite=Lax` mitiga parte del riesgo.	Agregar un token por sesión y validarlo en cada proceso POST.
Sin límite de intentos de login	No hay bloqueo temporal ni CAPTCHA ante intentos repetidos.	Limitar intentos por usuario/IP.
Cookie sin `secure`	La cookie de sesión no marca `secure` (el proyecto se ejecuta por HTTP local).	Activar `secure` y HTTPS en producción.
Credenciales por defecto	MySQL con `root` sin contraseña y cuenta `admin` de demostración con clave simple.	Crear usuario de BD con privilegios mínimos y eliminar/cambiar `admin`.
Contacto sin protección anti-spam	El formulario público puede recibir envíos masivos; los mensajes solo se leen en la base de datos.	Agregar CAPTCHA/límite y una vista de administración.
Mensaje por GET	El texto de `?mensaje=` se muestra escapado, pero cualquiera puede construir una URL con un texto arbitrario.	Usar mensajes «flash» en sesión en lugar de parámetros GET.
Errores de PHP	Si `display_errors` está activo, un acceso directo a un proceso interno puede mostrar avisos.	Desactivar `display_errors` en producción.
Sin recuperación de contraseña ni perfil	No existe cambio ni recuperación de clave.	Incluirlos en una versión evolutiva.

Los controles de acceso por rol y la auditoría de cambios no aplican en esta versión: no existen roles y las operaciones no se registran en bitácora.

10. Pruebas técnicas

Plan de pruebas derivado del comportamiento del código. La columna «Resultado esperado» proviene de la lectura del código; las columnas «Resultado obtenido» y «Estado» deben completarse al ejecutar cada prueba en el entorno instalado (estado inicial: **Pendiente**). Si una prueba falla, registrar el defecto, la corrección aplicada y la repetición.

ID	Prueba	Precondición	Pasos resumidos	Resultado esperado	Resultado obtenido	Estado
PT-01	Registro de usuario válido	Usuario/correo no existen	Registrarse con datos válidos	Redirige a login: «Cuenta creada correctamente…»; fila nueva en `usuarios` con hash	[Registrar]	Pendiente
PT-02	Registro con datos inválidos	Formulario de registro	Contraseña sin número; usuario duplicado; correo repetido	Mensaje de error específico; se conservan campos salvo contraseñas	[Registrar]	Pendiente
PT-03	Inicio de sesión válido	Cuenta `admin` activa	Ingresar con usuario o con correo	Abre Inventario; banner «Sesión activa: admin»	[Registrar]	Pendiente
PT-04	Inicio de sesión inválido	Cuenta existente	Clave incorrecta / usuario inexistente	Mismo mensaje en ambos casos; sin sesión	[Registrar]	Pendiente
PT-05	Acceso a página privada sin sesión	Sin sesión	Abrir `?pagina=inventario`	Redirige a login con «Debes iniciar sesión para continuar.»	[Registrar]	Pendiente
PT-06	Crear registro de inventario	Sesión iniciada	Detalle y cantidad válidos → Guardar	«Registro guardado en el inventario.»; aparece en la lista	[Registrar]	Pendiente
PT-07	Validación de cantidad	Sesión iniciada	Cantidad -1, 1000001 y «abc» (alterando el HTML)	Rechazo con «La cantidad debe ser un número entero entre 0 y 1.000.000.»	[Registrar]	Pendiente
PT-08	Editar registro propio	Registro existente	Editar → cambiar → Actualizar	«Registro de inventario actualizado correctamente.»	[Registrar]	Pendiente
PT-09	Eliminar registro	Registro existente	Eliminar → confirmar	«Registro eliminado del inventario.»; ya no aparece	[Registrar]	Pendiente
PT-10	Aislamiento entre usuarios	Dos cuentas con datos	Con la cuenta B abrir `inventario-editar&id=` de la cuenta A	«El registro no existe.»; no se muestran datos ajenos	[Registrar]	Pendiente
PT-11	Eliminar por GET	Sesión iniciada	Abrir `procesos/inventario/eliminarInventario.php?id=1` en la barra	No elimina; redirige a Inventario	[Registrar]	Pendiente
PT-12	CRUD de pedidos	Sesión iniciada	Crear, editar y eliminar un pedido	Mensajes de éxito correspondientes; datos actualizados en `pedidos`	[Registrar]	Pendiente

ID	Prueba	Precondición	Pasos resumidos	Resultado esperado	Resultado obtenido	Estado
PT-13	CRUD de proveedores	Sesión iniciada	Crear, editar y eliminar un proveedor	Mensajes de éxito correspondientes; datos actualizados en `proveedores`	[Registrar]	Pendiente
PT-14	Formulario de contacto	Sin sesión	Enviar nombre, correo y mensaje válidos	«Mensaje enviado exitosamente.»; fila en `mensajes_contacto`	[Registrar]	Pendiente
PT-15	Cierre de sesión	Sesión iniciada	Pulsar Salir; luego «Atrás» del navegador	«Sesión cerrada correctamente.»; no se muestran datos privados	[Registrar]	Pendiente
PT-16	Diseño responsivo	Sistema disponible	Abrir en ancho < 868 px (celular)	Hero en columna; listas apiladas; sin desbordes	[Registrar]	Pendiente
PT-17	Respaldo y restauración	BD con datos	`mysqldump` y restauración en BD vacía	Datos recuperados completos	[Registrar]	Pendiente
PT-18	Intento de inyección SQL	Pantalla de login	Usuario: `' OR '1'='1`	Sin acceso; mensaje de credenciales inválidas	[Registrar]	Pendiente
PT-19	Intento de XSS	Sesión iniciada	Detalle: `<script>alert(1)</script>`	Se muestra como texto; no se ejecuta	[Registrar]	Pendiente

Cada prueba ejecutada debe tener una evidencia numerada (captura o registro de la consulta SQL resultante).

11. Despliegue y puesta en funcionamiento

Paso	Actividad	Detalle
1	Preparar servidor	Instalar Apache 2.4, PHP ≥ 7.3 (recomendado 8.x) con `mysqli`/`mbstring`, y MySQL/MariaDB. Habilitar `AllowOverride` para que funcionen los `.htaccess`.
2	Copiar/publicar aplicación	Copiar la carpeta `sparrow/` a la raíz web (en XAMPP: `htdocs/sparrow`). `URL_BASE` se calcula sola.
3	Configurar variables	Editar `config/config.php` con servidor, usuario (no `root`), contraseña robusta y nombre de BD. Desactivar `display_errors` en `php.ini`.
4	Importar la BD	Importar `BD/sparrow_db.sql`; luego eliminar o cambiar la cuenta `admin` y los datos de ejemplo.
5	Configurar permisos	Crear un usuario MySQL con privilegios solo sobre `sparrow_db`; permisos de lectura para el servidor web sobre la carpeta del proyecto.
6	Iniciar servicios	Iniciar Apache y MySQL (panel de XAMPP o servicios del sistema).

Paso	Actividad	Detalle
7	Prueba de humo	Abrir inicio; registrarse; iniciar sesión; crear, editar y eliminar un registro; cerrar sesión.
8	Registrar versión	Anotar fecha, versión (1.0.0) y responsable en la sección 13.3.

11.1 Acceso desde dispositivos móviles

El sistema es una página web adaptable; no requiere aplicación móvil. Para probarlo desde un celular en la misma red local:

1. Averiguar la IP del equipo servidor (`ipconfig` en Windows).
2. Permitir Apache en el firewall de Windows para la red privada.
3. Abrir en el celular `http://<IP-del-servidor>/sparrow/`.

Debe registrarse la URL usada, la red, los navegadores probados y evidencia de al menos dos tamaños de pantalla. [Completar]

12. Respaldo y recuperación

Elemento	Definición
Frecuencia de respaldo	Antes de cada actualización o reimportación del script y, en uso real, de forma diaria o semanal. [Confirmar con el equipo]
Qué se respalda	Base de datos `sparrow_db`; carpeta del proyecto (código); `config/config.php` guardado de forma segura (contiene credenciales).
Ubicación	[Completar: disco externo, nube institucional, etc.]
Responsable	[Completar]
Prueba de restauración	[Completar fecha y resultado] (ver PT-17).

Procedimiento de respaldo

```
# Desde la consola (CMD) en Windows con XAMPP
C:\xampp\mysql\bin\mysqldump.exe -u root -p --databases sparrow_db >
respaldo_sparrow_AAAAMMDD.sql

# Respaldo del código
# Copiar la carpeta C:\xampp\htdocs\sparrow (sin publicar contraseñas reales)
```

Procedimiento de restauración

```
C:\xampp\mysql\bin\mysql.exe -u root -p < respaldo_sparrow_AAAAMMDD.sql

# Alternativa: phpMyAdmin → Importar → elegir el archivo de respaldo
```

> **Nota:** El archivo `BD/sparrow_db.sql` solo recrea la estructura y los datos de ejemplo; **no** restaura los datos reales de los usuarios. Para eso se necesita el respaldo con `mysqldump`.

13. Mantenimiento técnico

13.1 Mantenimiento preventivo

- Actualizar de forma controlada PHP, Apache y MySQL (XAMPP) y verificar el sistema después de cada actualización.
- Verificar que los respaldos se generen y puedan restaurarse.
- Revisar los registros de error de Apache y PHP (`C:\xampp\apache\logs\error.log`).
- Revisar la tabla `mensajes_contacto` y depurar mensajes antiguos o no deseados.
- Auditar cuentas de usuario y eliminar la cuenta de demostración en entornos reales.

13.2 Mantenimiento correctivo

Registrar el error (pantalla, pasos, mensaje), analizar la causa en la capa correspondiente (vista, proceso, validación o DAO), corregir, repetir las pruebas de la sección 10 relacionadas y actualizar la versión y este manual.

13.3 Mantenimiento evolutivo

Mejoras planeadas sugeridas: recuperación y cambio de contraseña, perfil de usuario, vista para leer mensajes de contacto, descuento automático de inventario al registrar pedidos, búsqueda y paginación en listas, tokens CSRF y límite de intentos de login, roles de usuario.

Para agregar o cambiar funcionalidades, el README del proyecto indica qué archivos modificar:

Quiero…	Base de datos	DAO	Lógica / proceso	Vista
Agregar un campo (ej. teléfono en proveedores)	`BD/sparrow_db.sql`	`proveedorDAO.php` (SQL y `bind_param`)	`validarProveedor.php`, `guardar/actualizarProveedor.php`	`formularioProveedor.php`, `formularioEditar.php`, `listaProveedores.php`
Cambiar una regla de validación	—	—	Archivo `logica/validar…`	—
Cambiar usuario o clave de MySQL	—	—	`config/config.php`	—
Cambiar colores o diseño	—	—	—	`css/estilos.css`
Agregar un módulo nuevo	Tabla nueva	DAO nuevo	Procesos y validación nuevos	Vistas nuevas + entrada en `$paginas` de `index.php`

Registro de cambios:

Fecha	Versión	Cambio	Responsable	Pruebas posteriores
2026	1.0.0	Versión inicial entregada del sistema SPARROW.	[Completar]	[Completar]
[ ]	[ ]	[ ]	[ ]	[ ]
[ ]	[ ]	[ ]	[ ]	[ ]

14. Solución de problemas frecuentes

Problema	Posible causa	Diagnóstico	Solución
No abre la aplicación	Apache detenido o puerto ocupado; carpeta mal ubicada	Verificar Apache en el panel de XAMPP; probar `http://localhost/`	Iniciar Apache; si usa otro puerto abrir `http://localhost:8080/sparrow/`; verificar que `index.php` esté en `htdocs/sparrow/`.
«No fue posible conectar con la base de datos…»	MySQL detenido, BD no importada o credenciales incorrectas	Revisar MySQL en el panel; abrir phpMyAdmin; revisar `config/config.php`	Iniciar MySQL, importar `BD/sparrow_db.sql` o corregir `USUARIO`/`PASSWORD`/`BASE_DATOS`.
MySQL tiene contraseña	`PASSWORD` vacío en la configuración	La conexión falla con acceso denegado	Escribir la contraseña en `PASSWORD` de `config/config.php`.
El usuario no puede ingresar	Credenciales erróneas o cuenta inexistente	Buscar el usuario en la tabla `usuarios`	Verificar datos; registrar una cuenta nueva. No hay recuperación automática.
CSS/imágenes no cargan o enlaces rotos	`URL_BASE` mal calculada (carpeta anidada o virtual host)	Inspeccionar la ruta de `css/estilos.css` en el navegador	Verificar la ubicación de la carpeta; evitar `sparrow/sparrow`.
No guarda información	Validación fallida o error de BD	Leer el mensaje rojo; revisar tablas y logs	Corregir el dato según la regla; verificar que las tablas existan.
Pantalla en blanco o errores de PHP	Versión de PHP antigua o extensión desactivada	Revisar `error.log` y `phpinfo()`	Usar PHP ≥ 7.3 y activar `mysqli`/`mbstring`.
La vista se desordena en celular	Caché del navegador o viewport	Probar en varias resoluciones	Recargar sin caché; revisar `@media` en `estilos.css`.

15. Documentación del código

Se seleccionan los fragmentos más representativos del sistema y se explica su función.

15.1 Autenticación

```
if ($_SERVER["REQUEST_METHOD"] !== "POST") {
    redirigir("login");
}

$error = validarLogin();
```

```
if ($error !== "") {
    recordarCampos(["txtLogin"]);
    redirigir("login", $error, "error");
}

$login    = trim($_POST["txtLogin"]);
$password = $_POST["txtPassword"];

// Busca el usuario por nombre de usuario o por correo.
$usuario = buscarUsuarioPorLogin($conexion, $login);

// password_verify compara la contraseña escrita con el hash guardado.
// El mensaje es el mismo si falla el usuario o la contraseña, para no dar pistas.
if (!$usuario || !password_verify($password, $usuario["password_hash"])) {
    recordarCampos(["txtLogin"]);
    redirigir("login", "Este usuario no está registrado o la contraseña es incorrecta.",
"error");
}

// Nuevo ID de sesión al autenticarse (evita la fijación de sesión).
session_regenerate_id(true);

$_SESSION["usuario_id"] = (int) $usuario["id"];
$_SESSION["usuario"]    = $usuario["usuario"];
$_SESSION["nombre"]     = $usuario["nombre"];

redirigir("inventario");
```

Fragmento 1. procesos/usuario/iniciarSesion.php.

Elemento	Contenido
Archivo y ruta	`procesos/usuario/iniciarSesion.php`
Lenguaje	PHP
Función/clase/método	Script de proceso (usa `validarLogin()`, `buscarUsuarioPorLogin()`)
Descripción	Valida el formulario, busca al usuario por nombre o correo, verifica la contraseña contra el hash, regenera el ID de sesión y guarda los datos del usuario en `$_SESSION`.
Entradas	`txtLogin`, `txtPassword` por POST.
Salidas	Redirección a `inventario` o a `login` con mensaje.
Excepciones/validaciones	Método distinto de POST; campos vacíos; usuario inexistente o clave incorrecta (mensaje único).

15.2 Conexión a base de datos

Ver el fragmento completo de `config/conexionBDD.php` en la sección 8.4. Parámetros: constantes `SERVIDOR`, `USUARIO`, `PASSWORD`, `BASE_DATOS`; manejo de error con `connect_errno` y `die()` con mensaje genérico; juego de caracteres `utf8mb4`.

15.3 Validación de datos (inventario)

```
function validarInventario()
{
    // Verifica que los campos hayan llegado por POST.
    if (!isset($_POST["txtDetalle"], $_POST["txtCantidad"])) {
        return "Por favor completa los dos campos.";
    }

    $detalle       = trim($_POST["txtDetalle"]);
    $cantidadTexto = trim($_POST["txtCantidad"]);

    // Se compara contra "" para no rechazar la cantidad 0 (empty() la rechazaría).
    if ($detalle === "" || $cantidadTexto === "") {
        return "Por favor completa los dos campos.";
    }

    if (mb_strlen($detalle, "UTF-8") < 3 || mb_strlen($detalle, "UTF-8") > 150) {
        return "El detalle debe tener entre 3 y 150 caracteres.";
    }

    // La cantidad debe ser un entero entre 0 y 1.000.000.
    $cantidad = filter_var($cantidadTexto, FILTER_VALIDATE_INT, [
        "options" => ["min_range" => 0, "max_range" => 1000000]
    ]);

    if ($cantidad === false) {
        return "La cantidad debe ser un número entero entre 0 y 1.000.000.";
    }

    return "";
}
```

Fragmento 2. logica/validarInventario.php.

Elemento	Contenido
Archivo y ruta	`logica/validarInventario.php`
Lenguaje	PHP
Función/clase/método	`validarInventario()`
Descripción	Comprueba que lleguen los campos, que no estén vacíos (comparación con `""` para aceptar la cantidad 0), la longitud del detalle y que la cantidad sea un entero entre 0 y 1.000.000.
Entradas	`$_POST["txtDetalle"]`, `$_POST["txtCantidad"]`.
Salidas	Cadena vacía si es válido; mensaje del primer error en caso contrario.
Excepciones/validaciones	`FILTER_VALIDATE_INT` con rango; `mb_strlen` para UTF-8.

15.4 Consulta segura y aislada por usuario

```
// Obtiene todos los registros del usuario.
// Devuelve un mysqli_result (o false si la consulta falla).
function listarInventario($conexion, $usuarioId)
```

```
{
    $sql = "SELECT id, detalle, cantidad
            FROM inventario
            WHERE usuario_id = ?
            ORDER BY id ASC";

    $stmt = $conexion->prepare($sql);
    if (!$stmt) return false;

    // i = integer
    $stmt->bind_param("i", $usuarioId);
    $stmt->execute();

    $resultado = $stmt->get_result();
    $stmt->close();

    return $resultado;
}
```

Fragmento 3. datos/inventario/inventarioDAO.php — listarInventario().

Elemento	Contenido
Archivo y ruta	`datos/inventario/inventarioDAO.php`
Lenguaje	PHP + SQL (MySQLi)
Función/clase/método	`listarInventario($conexion, $usuarioId)`
Descripción	Obtiene los registros del usuario con una consulta preparada; el filtro `usuario_id = ?` aísla la información de cada cuenta.
Entradas	Conexión MySQLi e ID del usuario.
Salidas	`mysqli_result` o `false` si falla `prepare()`.
Excepciones/validaciones	Si `prepare()` falla devuelve `false`; el proceso detiene la ejecución con un mensaje.

15.5 Control de acceso y enrutamiento

```
$pagina = $_GET["pagina"] ?? "inicio";

if (!is_string($pagina) || !isset($paginas[$pagina])) {
    $pagina = "inicio";
}

$mensaje = (isset($_GET["mensaje"]) && is_string($_GET["mensaje"])) ? $_GET["mensaje"] : "";
$tipo    = (($_GET["tipo"] ?? "") === "error") ? "error" : "exito";

$definicion = $paginas[$pagina];
$esPrivada  = !empty($definicion["privada"]);

// ------------------------------------------------------------------------
// 2) Control de acceso
// ------------------------------------------------------------------------
if ($esPrivada && !usuarioLogueado()) {
    redirigir("login", "Debes iniciar sesión para continuar.", "error");
}
```

```
if (!empty($definicion["invitado"]) && usuarioLogueado()) {
    redirigir("inventario");
}

// Cabeceras de seguridad básicas.
header("X-Content-Type-Options: nosniff");
header("X-Frame-Options: SAMEORIGIN");
```

Fragmento 4. index.php — lista blanca y control de acceso.

Elemento	Contenido
Archivo y ruta	`index.php`
Lenguaje	PHP
Función/clase/método	Bloque de enrutamiento (sin función)
Descripción	Toma `?pagina=`, la valida contra la lista blanca `$paginas`, obtiene el mensaje a mostrar, redirige si falta o sobra sesión y envía cabeceras de seguridad.
Entradas	`$_GET["pagina"]`, `$_GET["mensaje"]`, `$_GET["tipo"]`.
Salidas	Carga de proceso y vistas o redirección.
Excepciones/validaciones	Página no listada → `inicio`; privada sin sesión → `login`; invitado con sesión → `inventario`.

15.6 Registro de usuario

Ver `procesos/usuario/registrarUsuario.php` y `logica/validarUsuario.php` (`validarRegistro()`). El proceso sigue cinco pasos documentados en el propio código: validar, limpiar, verificar duplicados (`existeUsuario`, `existeCorreo`), cifrar con `password_hash()` e insertar con `insertarUsuario()`.

16. Control de versiones y entrega técnica

Entregable	Formato	Estado
Código fuente completo	`sparrow.zip` o repositorio	Disponible (`sparrow.zip`)
Base de datos / script	`BD/sparrow_db.sql`	Incluido en el código
Manual técnico	.docx	Este documento
Manual de usuario	.docx (documento separado)	Entregado aparte
Diagramas finales	Dentro del manual (Figuras 2, 3 y 5)	Incluidos
Evidencias de pruebas	Capturas / tabla de pruebas	Pendiente (sección 10)
Archivo de dependencias	No aplica (sin Composer ni npm)	No aplica

Entregable	Formato	Estado
Archivo README	`README.md`	Incluido

Si el equipo usa Git, se recomienda: rama principal estable, ramas por funcionalidad, mensajes de commit descriptivos y etiquetas de versión (`v1.0.0`). [Completar con el enlace del repositorio]

17. Lista de verificación antes de entregar

✔ = verificado al revisar el código · ☐ = pendiente de verificar por el equipo.

Estado	Criterio
✔	El manual corresponde al código de la versión 1.0.0 entregada.
✔	La arquitectura, la estructura de carpetas y los diagramas coinciden con el código.
✔	El diccionario de datos coincide con `BD/sparrow_db.sql`.
✔	No aparecen contraseñas reales ni datos sensibles (solo valores de ejemplo).
☐	Se indican las versiones exactas de PHP, MySQL y XAMPP instalados.
☐	El procedimiento de instalación fue probado desde cero.
☐	Las pruebas críticas (sección 10) se ejecutaron y su resultado quedó registrado.
☐	Se incorporaron capturas numeradas de instalación y pruebas.
☐	Se probó el respaldo y la restauración.
☐	Se completaron integrantes, repositorio y datos marcados como [Completar].

18. Anexos técnicos

Anexo	Contenido
A	Script SQL de creación de la base de datos: `BD/sparrow_db.sql` (secciones de `CREATE DATABASE` y `CREATE TABLE`).
B	Script SQL de datos iniciales: bloque `INSERT` del mismo archivo (usuario `admin`, 3 de inventario, 1 pedido y 1 proveedor).
C	Diagrama entidad-relación final: Figura 5.
D	Diagrama de componentes/arquitectura final: Figura 2. (No hay diagrama de clases: el proyecto no usa programación orientada a objetos.)
E	Diagrama de flujo de operaciones: Figura 3.
F	Evidencias completas de pruebas: [Completar al ejecutar el plan de la sección 10].

Anexo	Contenido
G	Estructura de carpetas: sección 7. Dependencias: ninguna.
H	Registro de versiones y cambios: sección 13.3.
I	Enlace al repositorio o medio de entrega del código fuente: `sparrow.zip`. [Completar con el repositorio, si existe]